

The 10th International Conference on Computer Science and Computational Intelligence 2025

Employee monitoring system with computer vision: Face and activity recognition

Nathanael Deciano Sugiharto^{a,*}, Darian Elbert^a, Ivan Sebastian Edbert^a

^aComputer Science Department, School of Computer Science, Bina Nusantara University, Jakarta 11840, Indonesia

Abstract

Vision-based surveillance system is used by major countries to monitor public spaces, namely face recognition to identify people and activity recognition to detect unwanted behaviour. The paper proposes a flexible and lightweight monitoring pipeline that runs both face and activity recognition at a time and is aimed for convenient deployment on closed spaces like schools or offices. The pipeline leverages popular vision models such as YOLOv8 for object detection, InsightFace for face recognition, and a vision-language model, CLIP, for zero-shot activity recognition that won't need any model training. Testing the accuracy is done using two office footages, each with different complexities of camera angle and crowd distribution, one being far more complex than the other. The result is that YOLOv8 peaked at 95,73% accuracy, InsightFace at 96,56% accuracy, and CLIP at 53,25%. The testing provided an insight that the pipeline's accuracy deteriorates at more complex footages with varying distances and angles of the object, and further research are needed on improving the pipeline's optimization, known faces processing, and camera positioning.

© 2025 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0>)

Peer-review under responsibility of the scientific committee of the 10th International Conference on Computer Science and Computational Intelligence (ICCSCI) 2025

Keywords: Computer Vision; YOLO; InsightFace; Object Detection; CLIP; Face Recognition; Activity Recognition; Workplace Monitoring; Zero-Shot Classification

1. Introduction

Usage of surveillance system using CCTVs has been growing in major countries. Cities such as Chongqing have planted 2,5 million CCTVs, while London has 500 thousand and Moscow with 170 thousand CCTVs installed as it is shown on Figure 1. Computer vision is applied to the system, enabling the system to perform detection and recognition tasks helping authorities monitor around the area. For example, detecting deviant behavior from criminals as a

* Corresponding author.

E-mail address: nathanael.sugiharto@binus.ac.id

preventive approach of handling crime (activity recognition), or to collect evidence and track wanted criminals through a face recognition system [1], [2].

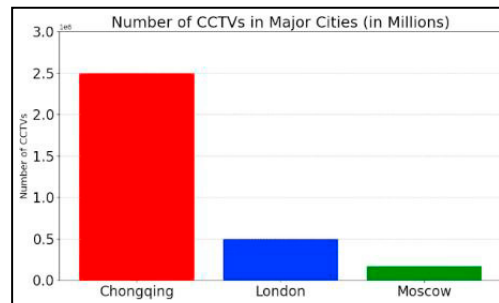


Fig. 1. Number of CCTVs in Major Cities

Furthermore, face/activity recognition could be used by organizations/institutions in closed spaces for monitoring or attendance, making operational processes more efficient [3]. Moreover, a study proves that even just 1% of AI penetration increase (including face recognition) in Chinese A-share-listed firms can improve total factor productivity into 14.2% [4]. Face recognition is generally chosen for biometric authentication because of its superiority. Its advantages are because of the difficulty of imitating human faces, and its quick process that requires no physical contact [5]. Deep learning models are popular for the task, for example, YOLO which is widely used for general object detection tasks and is already proven by its usage in many industries [6], [7]. As for face recognition tasks, there is InsightFace, which is a state-of-the-art face recognition framework that has been used in many cases including attendance systems [8], [9].

In addition to using vision model, a vision-language model like CLIP can be incorporated to recognize activities. The reason for using such a model is due to the extensive resource needed to train a specialized deep learning model for activity recognition. Hence, a model capable of zero-shot image classification like CLIP is needed for more convenient and flexible deployment in organizations [10], [11].

The proposed solution in this paper is a monitoring system that is lightweight and flexible for an organization. Utilizing face recognition along with activity recognition, the pipeline will receive frame from camera(s) and output the name of recognized people and their current activity. The frameworks involved are ensured to be free to access, which are the previously mentioned YOLO, InsightFace, and OpenAI's CLIP.

2. Literature Review

Generally, Object Detection processes are classified into 2 categories, two stage object detectors and single stage object detectors. A two-stage object detector starts by finding regions or areas that might contain objects and carefully check areas to find objects, while accurate, it also takes more time. Meanwhile, a single stage object detector skips the process of finding regions that might contain objects, it looks through the whole Image and detect objects from all possible locations, which is fast but may not be accurate. In recent years, You Only Look Once (YOLO) Models are presented as the solution, which is a single stage object detector that is highly accurate and fast [6], [7].

With the evolution of Artificial Intelligence, Minghao Wang proposed other enhancements upon YOLO. Changing YOLOv8's architecture with ConvNeXtV2 and DyHead which pulled a slightly better result than the base model of YOLOv8 meaning that YOLOv8 is versatile and can be improved upon it too [7]. Meanwhile, Zhongjie Huang et al. used YOLOv8 for traffic sign detection with its original architecture but modified the data augmentation pipeline by adding motion blur, fog simulation, and raindrop noise on the images to make the model susceptible to downgraded image quality, similar to what the model will detect later on. They also replaced the Neck of the architecture from Feature Pyramid Network (FPN) and Path Aggregation Network (PAN) to Asymptotic Feature Pyramid Network (AFPN) [12].

Studies for face recognition systems for monitoring purposes are already done, namely the study done by Shreyak Sawhney that combines Viola-Jones face detection with Eigenfaces embeddings for face recognition as a classroom attendance system [13]. Another usage in class attendance is done by Kuang and Baul but with the use of FaceNet

embeddings for face recognition. It successfully runs on an office laptop and received positive feedback from the users, assuring that facial recognition attendance system is convenient and useful [14].

Another research by Yohansen et al. implemented Residual Networks-50 (ResNet50) backbone (same as InsightFace). On a dataset of 53 students (13050 face data) and with 0,0001 learning rate and 100 epochs, it gains 99% accuracy in the test and train data [5]. ResNet50 for face recognition tasks has proven to be more effective than VGG16, LBP, VGG-Face, etc when tested on Yale B Database and CMU face Database. It gains 97,23% recognition rate in Yale B and 98,38% on CMU. [15]. Wangean et al. also implemented ResNet50 architecture on an attendance system. It peaks at 95% accuracy on test data, performing better when compared with LBPH method [16].

As for works regarding RetinaFace and Arcface implementation, a combination of both concepts is implemented on cattle face identification system by Bebei Xu et al., it achieves 91,3% accuracy at 24 frames per second (FPS). It performs better compared to Softmax (87,5%) and Cosface (80,3%) loss functions with same speed [17]. Thai-Viet Dang implemented ArcFace loss function on a Jetson Nano machine for smart home system, it achieves 94-97% accuracy at 21-25 FPS. Better than DLIB (52-59%, 9-10 FPS) and LBD (84-88%, 19-21 FPS) [18]. Manyuang Zhang et al. also made masked face recognition possible with ArcFace or CosFace as the loss function while also doing it on a large scale. This was possible due to their ability to clean the data properly with the technique called Iterative data noise cleaning which was done through WebFace42M. In the final model, the authors used CosFace instead of ArcFace because it is slightly better in performance and accuracy [19].

On the other hand, human activity recognition is a challenging field, especially in vision-based approach using 3D activity recognition. Usually, it represents the human body with skeletal points. While dataset for benchmarking like Human3.6M exists and the approach is proven to be effective, it is still unpopular as extensive resource is needed for processing the data. The reliability of handling general scenario remains a challenge [20], [21].

However, there have been studies on using deep learning approaches, namely the work done by Amurtha et al. for suspicious activity detection. They fine-tuned VGG-16 for feature extraction and used LSTM for sequence classification. It was tested on KTH and CAVIAR dataset, videos from YouTube, and self-obtained footage. The accuracy on training is 76% on 10 epochs, while the testing is 87,15% [22].

Open AI's CLIP (Contrastive Language-Image Pretraining), a machine learning model made to correlate text and image. CLIP stands out from other models due to its unique pre-training process called contrastive pre-trained, this made CLIP possible to output short descriptive text of an image [11]. A study is done by Bhat & Jain that shows CLIP still achieves 80,95% accuracy (Top 1) and 89,88% (Top 5), only about 3% behind face-recognition library's (at 92,27%) [10]. Other study by Qian and Hu proposed a framework that enables online label learning for CLIP to improve prediction distribution and proxy learning to solve modality gap between image and text. It achieved 78,94% accuracy on ImageNet [23]. Meanwhile, a work of improving CLIP is done by Ma et al. in which a Mixture of Data Experts framework using clustering is used for CLIP training to improve its effectiveness and efficiency, and it results in better performance on CLIP benchmarks, outperforming CLIP, openCLIP, and metaCLIP in both accuracy and computing cost [24].

3. Methodology

3.1. YOLO – Object Detection

You Only Look Once (YOLO) is a fast and accurate model due to its powerful deep learning architecture that implements Single-Shot Detector algorithm. YOLO detects objects by creating bounding boxes around it and adds labels according to its class [6]. Within every picture, YOLO makes a bounding box on objects that appear inside the image. For YOLO to form a grid, it predicts these parameters shown in Figure 2 below for a bounding box.

p_c	b_x	b_y	b_w	b_h	$p_{(c1)}$	$p_{(c2)}$	...	...	$p_{(cn)}$
-------	-------	-------	-------	-------	------------	------------	-----	-----	------------

Fig. 2 Parameters to make bounding boxes

With p_c representing probability of containing an object in the grid under the bounding box, b_x b_y representing the center of the predicting bounding box, b_w b_h representing the predicted dimension of bounding box, $p_{(c1)}$ $p_{(cn)}$ representing the conditional class probability the object belongs given the prior p_c .

A grid cell predicts $(B \times 5 + n)$ values, where B is the number of bounding boxes per grid cell. For each bounding box, Confidence score (C_c) is computed by $p_c \times \text{IoU}$ (Intersection over Union) between *ground-truth* and *predicted-bounding-box*. Then class specific scores (c_{ss}) are calculated for each bounding box.

Multiple bounding boxes could be made for the same object, due to adjacent grid cells predicting the same object, so those bounding boxes need to be discarded. By discarding those with low class scores, after discarding with this method another criterion for discarding is also used. Known as non max suppression, which uses the concept of Intersection over Union (IoU) done over two bounding boxes. By repeating these steps, the final bounding box is made [6].

3.1.1. YOLO's Functionality

YOLO works by first dividing an image/frame into $S \times S$ grid cells, each grid cell predicts B bounding boxes considering their positions and dimensions. In the center of that grid cell will lie an object that will be detected and put into the class. This grid cell is responsible for doing the task of detecting the object with the help of any suitable bounding box [6].

3.1.2. YOLOv8 Model Architecture

The YOLOv8 model architecture is composed of two major parts, the Backbone and head with each having a convolutional neural network. YOLOv8 model has a total of 23 layers consisting of 7 Convolutional layers, 8 C2f (bottleneck) layers, 4 concatenation layers, 1 Spatial Pyramid Pooling – Fast (SPPF) layer, 2 Up sampling layer, and 1 detection layer. Each of them serves their own purpose as seen at the model architecture (Table 1) K being Kernel, S for Stride, and P for Padding [7].

Table 1. YOLOv8 Model Architecture Layers

Layer	Type	Feature Map	K/S/P
1	Conv	$3 \rightarrow 16$	$3 \times 3 / 2 / 1$
2	Conv	$16 \rightarrow 32$	$3 \times 3 / 2 / 1$
3	C2f (Bottleneck)	$32 \rightarrow 32$	Various
4	Conv	$32 \rightarrow 64$	$3 \times 3 / 2 / 1$
5	C2f (Bottleneck)	$64 \rightarrow 64$	Various
6	Conv	$64 \rightarrow 128$	$3 \times 3 / 2 / 1$
7	C2f (Bottleneck)	$128 \rightarrow 128$	Various
8	Conv	$128 \rightarrow 256$	$3 \times 3 / 2 / 1$
9	C2f (Bottleneck)	$256 \rightarrow 256$	Various
10	SPPF	$256 \rightarrow 128$	Various
11	Upsample	-	2.0 Scale
12	Concat	-	-
13	C2f (Bottleneck)	$384 \rightarrow 128$	Various
14	Upsample	-	2.0 Scale
15	Concat	-	-
16	C2f (Bottleneck)	$192 \rightarrow 128$	Various
17	Conv	$64 \rightarrow 64$	$3 \times 3 / 2 / 1$
18	Concat	-	-
19	C2f (Bottleneck)	$192 \rightarrow 128$	Various
20	Conv	$128 \rightarrow 128$	$3 \times 3 / 2 / 1$
21	Concat	-	-
22	C2f (Bottleneck)	$384 \rightarrow 256$	Various
23	Detect	Various	Various

Focusing on Table 1, the convolution layers (layer 1, 2, 4, 6, 8, 17, and 20) extract low-level features such as edges, textures, and patterns from the input image. These layers are the basis of reducing spatial dimensions of the input image that also increases the depth, this helps finding more complex features at multiple levels [7].

Next are C2f (Bottleneck) layers (layer 3, 5, 7, 9, 13, 16, 19, and 22), common layers in modern CNNs, are used to reduce the parameters making the network computationally efficient, without removing any important features [7].

SPPF layer (layer 10) aggregates contextual information at multiple scales enabling object handling of varying sizes, which improves the robustness of object detection making it scale-invariant. It transforms 256 feature maps into 128 feature maps without leaving any crucial spatial information behind [7].

Upsample layers (layer 11 and 14) increases the resolution of the feature maps, enabling YOLO to detect smaller objects, while Concatenation layers (layer 12, 15, 18, and 21) combines feature maps from different levels of the network, which is used to do multi scale detection [7].

The Detect layer (layer 23) is used for the final object detection task where it makes predictions for bounding boxes and classes. These layers are formed in a way that looks like the visualization on Figure 3. Starting with the backbone, the process of extracting features from the input image happens. In addition, YOLOv8 upgraded YOLOv5 directly by using a new version of CSPDarknet53 architecture, replacing C3 modules with the newer C2f module, which can yield high-level features with contextual information. Finishing the feature extraction, the Head takes them to make bounding boxes, object classes, and object scores. YOLOv8 makes this an independent process, by creating the head to be detachable, this process allows branches to focus on its own tasks, improving the model's accuracy [25].

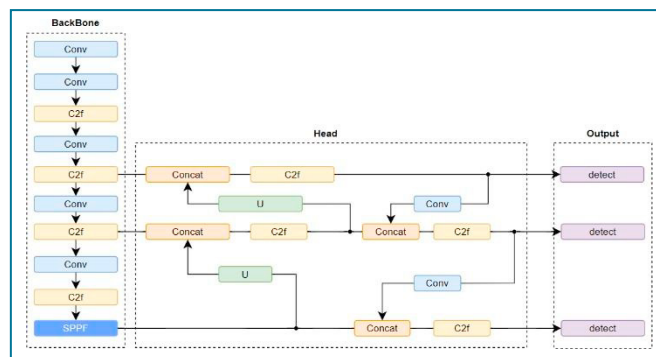


Fig. 3. YOLOv8 Architecture visualization

3.2. RetinaFace – Face Detection

RetinaFace is a face detection framework proposed by Jiankang Deng et al. that is a multi-task detection model with an efficient single-shot design. This framework combines bounding-box face detection along with 2D facial landmark localization, and 3D face reconstruction in a multi-task regression. Moreover, RetinaFace uses feature pyramid networks (FPN) as its backbone network for detecting features on multiple scales. That means, it is able to detect faces of varying sizes in a single image. It outputs a feature map with both low-level features like edges and high-level features like eyes, nose, etc. [26], [27]. This model is available in InsightFace package as the face detector that the proposed pipeline will use.

3.2.1. Bounding Box face Detection

This task is important to locate the boxes of faces inside the image. It uses anchor boxes in the image with various sizes that would be predicted by the model on its presence of face. It uses classification SoftMax loss and bounding box smooth L1 loss for its prediction [27].

3.2.2. 2D Facial Landmark Localization and 3D Face Reconstruction

RetinaFace captures facial features such as eyes, nose tips, and mouth corners obtained from the FPNs. It uses a point regression loss for landmark predictions, and the coordinates produced are relative to the bounding box [19]. On top of that, RetinaFace generates 1000 vertices in a 3D plane that resembles a face. The features are also obtained from the FPNs that are passed into another loss function (vertex loss and edge loss), resulting in a mesh of vertices shaping like a face. It is an important stage to help the framework capture faces with extreme angles in real-world

conditions. With a straightforward pipeline design and robust tasks, it creates a reliable, state-of-the-art face detection framework that also performs well in terms of speed [26], [27].

3.3. ResNet 50

ResNet is a deep CNN architecture proposed by He et al. from Microsoft, which introduced residual learning to handle degradation on deeper networks. It has been tested on various datasets and has been proven to perform better VGG. Its key idea is to use “residual blocks” with Formula 1 shown below:

$$H(x) = F(x) + x \quad (1)$$

$H(x)$ represents the output, $F(x)$ is the residual mapping, and x as the input. This would ease optimization because the ideal output ($H(x)$) would be close to x , and that makes $F(x)$ relatively small to prevent vanishing gradient. It also enables shortcut connection which the model can skip learning a mapping if it is zero ($F(x) = 0$).

On this case, ResNet 50 (50 layers) is used as it provides good balance between accuracy and computing cost. It implements 3 layers bottleneck block that reduces the number of channels on feature maps before expanding it again. The purpose of this is to reduce computational cost while the model could still learn complex features [28].

3.4. ArcFace – Loss Function

ArcFace or Additive Angular Margin Loss is another method proposed by Jiankang Deng Et al. that is aimed to optimize training for deep learning models. The authors modified softmax loss function (Formula 2) into having angular margin penalties, as shown below.

$$L = -\frac{1}{N} \sum_{i=1}^N \log \frac{e^{W_{yi}^T x_i + b_{yi}}}{\sum_{j=1}^N e^{W_j^T x_i + b_j}} \quad (2)$$

$$L = -\frac{1}{N} \sum_{i=1}^N \log \frac{e^{(s \cos(\theta_{yi} + m))}}{e^{s(\cos(\theta_{yi} + m))} + \sum_{j=1, j \neq yi}^N e^{s \cos \theta_j}} \quad (3)$$

As seen from Formula 2 and 3, The authors have set the bias (b) from into 0 and turned the weight (W) into 1. The logit function is changed into a dot product that enables angles with *cosine*. A margin (m) is added into the angle. Lastly, the feature (x_i) is normalized and rescaled into radius (s).

Those changes make sure the embeddings that are produced by the model improve faces representation. It enables more compactness within images in the same class (intra-class) and separability of different classes (inter-class), making a more discriminative and accurate classifier for face recognition [29], [30].

The ArcFace embedding generation is available within InsightFace python package. The model will be used to turn the images of known faces into embeddings, as well as input images that would be compared with the known faces using cosine distance [29].

3.5. CLIP – Vision Language Model

CLIP (Contrastive Language-Image Pre-training), a model which tells images and short text descriptions has correlation and belongs together. CLIP differs from Neural Networks (NN) where NN takes images, and text classes and gives probabilities to all text classes in the dataset depending on what image it is currently processing. Meanwhile CLIP is not restricted to the text class dataset, nor is it to classify an image to a word, instead CLIP uses short text descriptions describing what is presented to it. CLIP is trained on 400 million pairs of text and images, with each of them having to be processed in the same contrastive pre-training process [11].

3.5.1. Contrastively pre-trained

Unlike the usual CNN, CLIP was pre-trained contrastively. A batch of N image with text pairs are used for the pre-training process, these pairs of text and images are then used by CLIP to predict which of the N^2 possible pairs occurred. CLIP uses a multi-modal embedding space where it jointly trains an image encoder and a text encoder. With this CLIP is also computationally efficient because both encoders can be run simultaneously enabling parallel computation. With both encoders ready, CLIP uses cosine similarity and finds the maximum value of the N pair while minimizing cosine similarity of the $N^2 - N$ incorrect pairing, this method enables CLIP to do Zero-Shot Predictions. A visualization of this process can be seen in Figure 4 below [11].

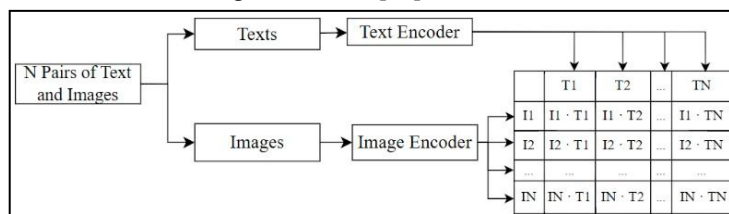


Fig. 4. CLIP's Contrastive pre-trained model

3.5.2. Zero-Shot Predictions

CLIP classifies images in Zero-Shot manner, which enables it to make predictions on images without it having prior explicit training on the image. To achieve this, it is needed for a set of prompts to be given, consisting of what the model should predict and become its parameter on which it needs to look out for in an image. Then those prompts will be encoded into vectors, and it will be used to predict the best text for the given image [11].

3.6. Proposed Pipeline

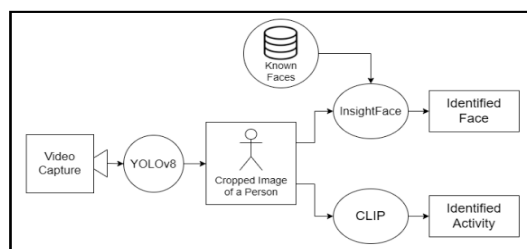


Fig. 5. Flowchart of Proposed Pipeline

Figure 5 above is the visualization of our proposed pipeline's flow. Before starting the pipeline, a directory of known faces must be prepared. It consists of folders named with each person's name, and inside those folders are a picture of the respective person's face. Those images would be turned into image embeddings and stored in a dictionary for the face recognition task.

Now, when the system is initialized, a frame from a video capturing device, such as CCTV or webcam, is received. Then an object detection model, YOLOv8, is used to locate each person in the frame. YOLOv8 will retrieve the bounding box of each "person" detected using 4 coordinates, and each detected "person" would be cropped into individual images to be passed into the next steps.

InsightFace, is the framework that is chosen for face recognition task. It uses RetinaFace for face detection, and a ResNet50 model with ArcFace loss for face recognition. It detects the face in every individual cropped image, then obtains 4 coordinates of the face' box, and the face bounding box would be cropped again from the image to be transformed into an embedding. That embedding will then be compared with each embedding of known faces, and the one with less distance is chosen as the identification (name).

On the other side, CLIP is initialized with a set of prompts, which are image descriptions such as "working on a laptop", "sleeping on the desk", etc. that are common workplace activities. After receiving a cropped-individual-

person image, CLIP will encode it and measure its similarity with each of the prompts. Then the prompt with the highest score will be picked as that person's activity.

3.7. Dataset & Testing

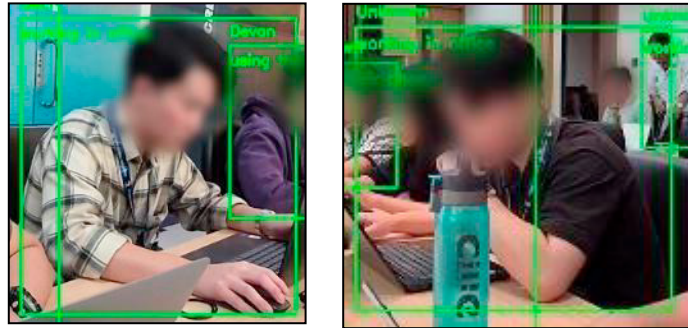


Fig. 6 (a) Example frame from video 1: (b) Example Result frame from video 2

The data that will be used for testing the performance of the pipeline is a real-word footage of an office environment that is obtained independently (primary data). The data is in the form of 2 videos featuring different angles of the office environment. The videos are both around 30 minutes, with the resolution of 720p, and both have a combination of natural lighting and lamps. The prompts set for CLIP are the same for each video, such as "working in office", "using their phone", "sleeping on desk", and "eating snack" [31]. Examples of which is shown in Figure 6.

The data will be labelled with manual annotations for each frame to assign ground truth for both faces and activities that are being detected and classified.

4. Results and Discussions

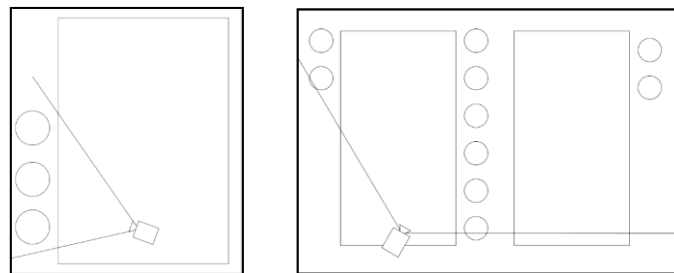


Fig. 7. (a) Video 1 Angle Visualization: (b) Video 2 Angle Visualization

The first video's camera position is like shown in Figure 7 (a), which is towards people sitting at a side of a table.

The composition of people is generally clearer and less complex, having only one line of people being focused on. YOLOv8 successfully detected 95,73% of the targets. The accuracy of face recognition (InsightFace) reached 96,58% on faces in this video. On the same video, CLIP successfully classified every action from the prompts set, but it is too sensitive with small changes in the people's pose and expression, such as a blinking person is considered "sleeping on desk", and people with their hands near the mouth considered as "eating snack". Hence the accuracy is only 53.25%.

Moving on to the second angle shown in Figure 6 (b), the camera is put on the end of a main table but angled at people on both sides of it. In addition, people on the other, farther table are visible, behind the people on the right side of the main table (distance is about 11 meters). The video contains more sideways angles to the faces, more obstruction, and farther targets to detect and recognize. Hence YOLOv8 successfully detected 48,15% of the people, while face recognition accuracy is only 29,67%, and the activity recognition is 33,44%.

Table 2. Accuracy Summary

Model	Video 1	Video 2
YOLOv8	95,73%	48,15%
InsightFace	96,56%	29,67%
CLIP	53,25%	33,44%

Table 2 shows the accuracy of each module on both experiments. It is apparent that both YOLOv8 and InsightFace's accuracy in a simpler environment is noticeably high. However, YOLOv8's detection performance is highly affected by the distance. Same goes for face recognition's accuracy that decreases at extreme angle and distance of the face. Additionally, the cropped image that YOLOv8 produces may contain multiple people, hence the face recognition performance becomes inconsistent. The face database also only provides frontal-view of each person's face, hence InsightFace lacks the reference of people's face from different angles.

Meanwhile, the activity recognition with CLIP is very sensitive to subtle changes of a person's pose and expression. These also include people that are obstructed or far away from the camera. On the bright side, its sensitivity is strong proof that CLIP could tell the presence of something accurately and is a potentially effective action descriptor if a more specific set of prompts is prepared.

A better setup on the external environment like camera placement may help the pipeline to perform better. For example, the cameras are preferably pointed towards frontal view of the people's faces, and the cameras should be angled in a way to prevent YOLOv8 capturing multiple, overlapping people inside the cropped image. Dedicating more cameras could also be done to monitor people on the farther side of the environment.

The pipeline reached 4 FPS while running on a laptop with Ryzen 7 4800H CPU and 16 GB RAM. While it is run without GPU acceleration, all three tasks are available for GPU utilization to improve its performance on further deployment.

5. Conclusion

In conclusion, a pipeline for a lightweight and free-to-use monitoring system which includes face and activity recognition pipeline is proposed and tested on footages of real-world closed-space environments with 2 different angles and complexity. The result showed that the pipeline performs well in a less complex footage but achieved low accuracy on a more complex footage. The result proves that the pipeline is highly affected by the complexity of the objects (people) in the footage, such as the diverse and overlapping angle, position, and distance of each person in which the complex scenario made YOLOv8 capture multiple people in a single cropped image, impacting the face recognition model on detecting and identifying the face. Thus, a better setup of cameras is needed to ensure good results. Additionally, the face database referenced by InsightFace lacks angle diversity for face, lowering its accuracy on recognizing side-profiles. Moreover, the set of prompts given to CLIP is not specific enough, resulting in overly sensitive predictions when subtle changes occur. Besides the accuracy and limitations, the pipeline has successfully run on a consumer grade laptop while retaining sufficient FPS, suggesting that the pipeline is viable to be used flexibly by organizations without needing high-end device. Future work should be aimed at improving, validating, and gaining more insights from the pipeline. It could start with the development of a more efficient face database system that would speed up the process of comparing detected faces with the known faces. Experiments to adjust the threshold of the models can help filter out low-confidence predictions and enhance their efficiency. Lastly, testing GPU utilization for the pipeline can give insights about the impact to the pipeline's performance.

6. Data Availability

The snapshot of the testing data is available at <https://github.com/DH3155/MonitoringSystemData>. The original footage is not available due to the confidential restriction of the company where the footage is recorded.

7. Author Contribution Statement

The credit to authorship contributions are as follows: Nathanael Deciano Sugiharto: Conceptualization, Software, Methodology, Data Curation, Writing – Original Draft. Darian Elbert: Methodology, Data Curation, Validation, Writing – Original Draft. Ivan Sebastian Edbert: Validation, Supervision, Project Administration.

References

- [1] I. Ryabchikov, N. Teslya, and N. Druzhinin, "Integrating Computer Vision Technologies for Smart Surveillance Purpose," in *2020 26th Conference of Open Innovations Association (FRUCT)*, 2020, pp. 392–401. doi: 10.23919/FRUCT48808.2020.9087479.
- [2] R. Socha and B. Kogut, "Urban video surveillance as a tool to improve security in public spaces," *Sustainability (Switzerland)*, vol. 12, no.

- 15, Aug. 2020, doi: 10.3390/su12156210.
- [3] Y. Patil Vishal and Jadhav, “Office Monitoring and Surveillance System,” in *Techno-Societal 2020*, R. and R. B. P. and S. S. B. and V. A. S. and M. B. Pawar Prashant M. and Balasubramaniam, Ed., Cham: Springer International Publishing, 2021, pp. 3–11.
- [4] X. Gao and H. Feng, “AI-Driven Productivity Gains: Artificial Intelligence and Firm Productivity,” *Sustainability (Switzerland)*, vol. 15, no. 11, Jun. 2023, doi: 10.3390/su15118934.
- [5] Y. Pratama, L. M. Ginting, E. H. L. Nainggolan, and A. E. Rismanda, “Face recognition for presence system by using residual networks-50 architecture,” *International Journal of Electrical and Computer Engineering*, vol. 11, no. 6, pp. 5488–5496, Dec. 2021, doi: 10.11591/ijece.v11i6.pp5488-5496.
- [6] T. Diwan, G. Anirudh, and J. V. Tembhumne, “Object detection using YOLO: challenges, architectural successors, datasets and applications,” *Multimed Tools Appl*, vol. 82, no. 6, pp. 9243–9275, 2023, doi: 10.1007/s11042-022-13644-y.
- [7] M. Wang, “Comparative Analysis of YOLOv8 Architectural Variations Based on Detection of Cow Stall Labels,” Mar. 2023, doi: 10.13140/RG.2.2.16044.03206.
- [8] J. Pu, L. Zhang, J. Zhang, Z. Xue, X. Zhang, and J. Yang, “Research on Classroom Attendance System Based on Face Recognition,” *J Phys Conf Ser*, vol. 1616, no. 1, p. 12095, Aug. 2020, doi: 10.1088/1742-6596/1616/1/012095.
- [9] K. G. O. -, E. J. I. -, J. O. O. -, and P. R. B. -, “AI based Face Recognition Attendance System,” *International Journal of Innovative Research in Engineering & Multidisciplinary Physical Sciences*, vol. 12, no. 2, Apr. 2024, doi: 10.37082/IJIRMP.v12.i2.230598.
- [10] A. Bhat and S. Jain, “Face Recognition in the age of CLIP & Billion image datasets,” Jan. 18, 2023, doi: <https://doi.org/10.48550/arXiv.2301.07315>.
- [11] A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” 2021. [Online]. Available: <https://arxiv.org/abs/2103.00020>
- [12] Z. Huang, L. Li, G. C. Krizek, and L. Sun, “Research on Traffic Sign Detection Based on Improved YOLOv8,” *Journal of Computer and Communications*, vol. 11, no. 07, pp. 226–232, 2023, doi: 10.4236/jcc.2023.117014.
- [13] S. Sawhney, K. Kacker, S. Jain, S. N. Singh, and R. Garg, “Real-Time Smart Attendance System using Face Recognition Techniques,” in *2019 9th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*, 2019, pp. 522–525. doi: 10.1109/CONFLUENCE.2019.8776934.
- [14] W. Kuang and A. Baul, “A real-time attendance system using deep learning face recognition,” in *ASEE Annual Conference and Exposition, Conference Proceedings*, American Society for Engineering Education, Jun. 2020, doi: 10.18260/1-2--33949.
- [15] B. R. Ilyas, B. Mohammed, M. Khaled, and K. Miloud, “Enhanced Face Recognition System Based on Deep CNN,” in *2019 6th International Conference on Image and Signal Processing and their Applications (ISPA)*, 2019, pp. 1–6. doi: 10.1109/ISPA48434.2019.8966797.
- [16] D. A. Wangean, G. Pangestu, S. Setyawan, F. I. Maulana, E. P. Gunawan, and C. Huda, “The implementation of ResNet-50 architecture for face recognition algorithm in attendance system,” *AIP Conf Proc*, vol. 2927, no. 1, p. 060014, Mar. 2024, doi: 10.1063/5.0205236.
- [17] B. Xu et al., “CattleFaceNet: A cattle face identification approach based on RetinaFace and ArcFace loss,” *Comput Electron Agric*, vol. 193, p. 106675, 2022, doi: <https://doi.org/10.1016/j.compag.2021.106675>.
- [18] T. V. Dang, “Smart home Management System with Face Recognition Based on ArcFace Model in Deep Convolutional Neural Network,” *Journal of Robotics and Control (JRC)*, vol. 3, no. 6, pp. 754–761, Nov. 2022, doi: 10.18196/jrc.v3i6.15978.
- [19] M. Zhang, B. Ma, G. Song, Y. Wang, H. Li, and Y. Liu, “Towards Large-scale Masked Face Recognition,” Oct. 2023, [Online]. Available: <http://arxiv.org/abs/2310.16364>
- [20] J. Wang et al., “Deep 3D human pose estimation: A review,” *Computer Vision and Image Understanding*, vol. 210, p. 103225, 2021, doi: <https://doi.org/10.1016/j.cviu.2021.103225>.
- [21] A. Gupta, K. Gupta, K. Gupta, and K. Gupta, “A Survey on Human Activity Recognition and Classification,” in *2020 International Conference on Communication and Signal Processing (ICCSPP)*, 2020, pp. 915–919. doi: 10.1109/ICCSPP48568.2020.9182416.
- [22] C. V. Amrutha, C. Jyotsna, and J. Amudha, “Deep Learning Approach for Suspicious Activity Detection from Surveillance Video,” in *2020 2nd International Conference on Innovative Mechanisms for Industry Applications (ICIMIA)*, 2020, pp. 335–339. doi: 10.1109/ICIMIA48430.2020.9074920.
- [23] Q. Qian and J. Hu, “Online Zero-Shot Classification with CLIP.” [Online]. Available: <https://github.com/ldstev/OnZeta>.
- [24] J. Ma et al., “MoDE: CLIP Data Experts via Clustering,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.16030>
- [25] M. Sohan, T. Sai Ram, and Ch. V. Rami Reddy, “A Review on YOLOv8 and Its Advancements,” 2024, pp. 529–545. doi: 10.1007/978-981-99-7962-2_39.
- [26] J. Deng, J. Guo, Y. Zhou, J. Yu, I. Kotsia, and S. Zafeiriou, “RetinaFace: Single-stage Dense Face Localisation in the Wild,” 2019. [Online]. Available: <https://arxiv.org/abs/1905.00641>
- [27] J. Deng, J. Guo, E. Ververas, I. Kotsia, and S. Zafeiriou, “RetinaFace: Single-Shot Multi-Level Face Localisation in the Wild,” in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 5202–5211. doi: 10.1109/CVPR42600.2020.00525.
- [28] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” 2015. [Online]. Available: <https://arxiv.org/abs/1512.03385>
- [29] J. Deng, J. Guo, J. Yang, N. Xue, I. Kotsia, and S. Zafeiriou, “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” *IEEE Trans Pattern Anal Mach Intell*, vol. 44, no. 10, pp. 5962–5979, Oct. 2022, doi: 10.1109/tpami.2021.3087709.
- [30] Y. Jie, M. Jiong, S. Baiyi, L. Mingxin, W. Shu, and W. Zhiyou, “Face recognition of students based on ArcFace,” in *Proceedings of the 2020 International Conference on Aviation Safety and Information Technology*, in ICASIT 2020. New York, NY, USA: Association for Computing Machinery, 2020, pp. 678–681. doi: 10.1145/3434581.3434712.
- [31] N. D. Sugiharto and D. Elbert, “Employee Monitoring System with Computer Vision: Face and Activity Recognition Dataset.” Accessed: Apr. 28, 2025. [Online]. Available: <https://github.com/DH3155/MonitoringSystemData>